

模型蒸馏核心 Gotchas

从“Teacher 输出做 SFT” 到可验证的 Distillation Data Engine

核心判断

蒸馏的主要风险不在“是否能把 Teacher 输出喂给 Student”，而在于数据分布、可学习性、验证、轨迹因果性、遗忘与真实部署评测。一个看似成功的蒸馏实验，很容易只学到 Teacher 的风格、格式或 benchmark 模板。

本手册覆盖	重点
17 类核心 Gotchas	Teacher / Data / Student / Reasoning / Agent / Evaluation / Deployment
P0-P2 风险分级	明确哪些问题会直接让实验结论失效
工程化闭环	Hard Example Mining + Verifier + Curriculum + Production-like Eval
上线前检查清单	从数据、训练到量化部署的可执行 Gate

版本：2026-08 | 适用：LLM / Reasoning / Coding / Tool-use / Agent 蒸馏

01 执行摘要

把蒸馏看成一个“受验证的数据生成与能力迁移系统”，而不是一次普通 SFT。

最重要的公式

$\text{Distillation Quality} \approx \text{Teacher Quality} \times \text{Data Coverage} \times \text{Verification} \times \text{Student Learnability} \times \text{Evaluation Quality}$ 。它更像乘法而不是加法：任何一项接近 0，整体结果都可能失效。

优先级	Gotcha	为什么危险
P0	Teacher 错误被当作 Ground Truth	随机错误会被训练成系统性偏差
P0	Synthetic Distribution $\neq$ Production Distribution	benchmark 好看但线上输入分布一变就崩
P0	Benchmark Contamination	会制造“能力大幅提升”的假象
P0	Agent Future-Observation Leakage	训练时偷看未来 observation，线上无法复现
P0	Student Capacity / Learnability 不足	蒸馏的是输出形式而非真正能力
P1	Raw CoT / 只蒸馏 Final Answer	可能学会冗长解释或猜答案，而非决策过程
P1	Catastrophic Forgetting / Safety Regression	领域能力上涨但通用能力与对齐退化
P1	Survivorship Bias	只看成功轨迹，失败恢复能力几乎为零
P2	Sampling / Duplicate / Tokenizer / Quantization	常被忽略，但会显著影响最终线上表现

一句话结论

真正有竞争力的蒸馏系统，核心逐渐从 Knowledge Distillation 转向 Distillation Data Engine：持续发现“Student 不会、Teacher 会、且能够被验证为正确”的样本。

02 统一心智模型

把蒸馏拆成 Teacher、Data、Student、Verifier、Evaluation 五个耦合系统。

层	关键问题	典型失败
Teacher	Teacher 的答案和过程是否可靠？	幻觉、错误工具调用、脆弱 reasoning
Data	数据是否覆盖真实流量与困难边界？	synthetic 偏置、重复、难度失衡
Student	Student 是否具备承载能力？	容量不足、tokenizer 不匹配、遗忘
Verifier	错误能否被发现并过滤？	无执行验证、reward hacking、错误标签

层	关键问题	典型失败
Evaluation	评测是否接近真实部署？	benchmark 泄漏、FP16 假象、线上分布偏移

工程原则

能执行验证就不要只相信 Teacher；能从真实流量抽样就不要只靠随机 prompt；能做 disagreement mining 就不要平均地蒸馏所有样本。

03 Teacher 与 Student 的能力边界

1. Teacher 强 $\neq$ Student 一定能学会 [P0]

Teacher 能力、Student 容量和可学习性之间存在硬约束。70B Teacher 的复杂规划轨迹，7B Student 可能只能学到表面模板。

诊断信号	修复策略
训练 loss 很快下降但 held-out 能力不涨；输出风格接近 Teacher，任务成功率变化小；长链任务在前几步后迅速崩	做能力边界探针与小规模 pilot；采用 curriculum 或中间尺寸桥接；按任务复杂度选择蒸馏目标，而非全量照搬

验证指标：按难度分桶后的 success rate；Student-Teacher capability gap；长度敏感曲线

2. Teacher 错误会被 Student 系统化 [P0]

Teacher 的随机 hallucination 一旦进入训练集，会从“偶发错误”变成稳定偏差。尤其在代码、数学、SQL、工具调用中风险极高。

诊断信号	修复策略
相同错误 pattern 在 Student 中重复出现；Teacher 错误率不高但 Student 对某类问题稳定出错；训练数据无法追溯验证状态	代码执行 / unit test / compiler 验证；数学做 symbolic 或 numeric check；Agent 使用环境结果与状态约束验证

验证指标：verified pass rate；false-positive label rate；错误 pattern 重复率

3. Single-Teacher Bias [P1]

单一 Teacher 会把自己的风格、盲区、知识缺口和拒答策略全部复制给 Student。蒸馏越充分，这种偏置越稳定。

诊断信号	修复策略
Student 与 Teacher 在同一问题上高度同错；风格和措辞异常一致；某些类别长期没有多样答案	多 Teacher / 多采样 / 多 verifier；对重要类别构造对抗样本；保留真实数据与人工金标

验证指标：error correlation；teacher diversity；answer entropy

4. Temperature 与 Sampling 的两难 [P2]

temperature 太低会导致 mode collapse；太高则会大量引入低质量 reasoning。单次生成通常不足以代表 Teacher 的可用能力。

诊断信号	修复策略
数据措辞高度模板化；同一 prompt 只有一个固定解法；高温后 verifier 拒绝率激增	N-sample + verifier + best-of-N；保留少量高质量 diverse positives；按任务类别单独调采样策略

验证指标： pass@N；diversity；verifier acceptance rate；unique n-gram / semantic clusters

5. Teacher 的长 CoT 不等于好监督 [P1]

Raw CoT 常包含冗余探索、事后合理化、错误分支和无因果价值的解释。直接 SFT 容易训练出“更啰嗦”而不是“更会推理”。

诊断信号	修复策略
token 数明显上涨但 accuracy 不涨；Student 固定使用冗长模板；决策点错误率没有下降	压缩轨迹，只保留关键决策状态；过滤错误分支与重复解释；针对过程做 step-level verifier

验证指标： accuracy/token；decision-point accuracy；trajectory compression ratio

04 数据分布与样本工程

6. Synthetic Distribution 与 Production Distribution 偏移 [P0]

合成 prompt 通常更完整、更干净、更合作；真实用户输入却可能短、乱、冲突、缺上下文、混合语言并伴随工具失败。

诊断信号	修复策略
离线 benchmark 很高但线上满意度低；对 typo / 模糊指代 / 长上下文敏感；真实流量中的失败类别在训练集中几乎没有	从真实流量统计输入分布再合成扩展；建立长度、语言、噪声、模糊度、工具状态分桶；做 production replay / shadow eval

验证指标： production-like success rate；分布距离；bucket coverage

7. 数据过于完美导致“任何问题都必须回答” [P1]

如果训练集永远是 question→perfect answer，Student 很难学会信息不足、不可解、应追问、应使用工具或应明确不确定的情况。

诊断信号	修复策略
hallucination 增加；缺信息时仍强行回答；工具本应调用时直接猜结果	加入 unanswerable / ambiguous / tool-required / missing-info 样本；显式训练 abstention 与 clarification 行为；把“不回答”也纳入可验证目标

验证指标: abstention calibration; tool-call precision/recall; hallucination rate

8. Random Distillation 浪费大量算力 [P1]

Student 已经会的样本信息增益很低。真正高价值区域是 Teacher 正确、Student 错误，或 Student 置信度异常的样本。

诊断信号	修复策略
训练集大量 easy samples; loss 降但关键 benchmark 停滞; 重复训练简单题	Student-Teacher disagreement mining; hard example mining; active distillation 与动态 curriculum

验证指标: gain per 1M tokens; disagreement resolution rate; hard-bucket uplift

9. Duplicate / Near-duplicate 造成权重失真 [P2]

大规模 synthetic generation 极易产生模板级近重复，导致某些 pattern 被过度训练，同时降低有效样本多样性。

诊断信号	修复策略
unique ratio 低; embedding cluster 极度集中; 某些回答模板出现频率异常高	exact / fuzzy / semantic dedup; 按 cluster 限流; 对高频模板做 reweighting

验证指标: effective sample size; cluster concentration; duplication rate

10. Benchmark Contamination 与模板泄漏 [P0]

Teacher 可能见过 benchmark，合成数据也可能包含 benchmark 的模板、答案或近义改写，从而形成闭环泄漏。

诊断信号	修复策略
公开 benchmark 提升远大于私有集; 相似题表现异常好; 训练集与 benchmark 有高语义相似样本	exact / fuzzy / semantic contamination scan; 建立 holdout private benchmark; 追踪数据 provenance

验证指标: contamination hit rate; private/public gap; template overlap

05 Reasoning、Logit 与 Tokenizer Gotchas

11. 只蒸馏 Final Answer 容易得到“会猜答案”的模型 [P1]

复杂任务的能力来自计划、状态更新、工具交互和中间决策。只监督最终答案，会丢失真正决定成功率的过程结构。

诊断信号	修复策略
一步 QA 提升明显，长链任务提升很小; 工具调用顺序不稳定; 中间状态错误率高	对关键决策点监督; trajectory / process distillation; 使用 step-level rewards 或 verifiers

验证指标: step success; trajectory success; final-only vs process ablation

12. Tokenizer / Vocabulary 不一致破坏 Logit Distillation [P2]

Teacher 与 Student tokenizer 不一致时，token-level probability 不处于同一空间，KL 蒸馏无法直接对齐。

诊断信号	修复策略
同一文本 token 边界差异巨大；代码/多语言 token 效率差异明显；logit mapping 不稳定	跨 tokenizer 优先 sequence-level distillation；必要时做 vocabulary mapping / shared tokenizer；先评估 token efficiency

验证指标： tokens/character；alignment coverage；cross-tokenizer loss stability

13. Logit Temperature 过低或过高 [P2]

温度过低时 Teacher 分布近似 one-hot，退化成普通 SFT；过高时分布过平，暗知识被噪声稀释。

诊断信号	修复策略
KL 项几乎无贡献；Student 分布校准没有改善；不同 T 的结果高度敏感	在大规模验证集上搜索 T 与 α ；关注 dark knowledge 与 calibration；按任务类型区分 temperature

验证指标： ECE / Brier score；KL contribution；accuracy-calibration frontier

06 Agent 蒸馏的特有高风险问题

Agent 不是“多轮文本 SFT”。轨迹数据必须遵守严格的因果结构。

14. Future-Observation Leakage [P0]

训练 action_t 时如果样本中泄漏了未来 observation，Student 学到的是无法在线获得的信息。这是轨迹蒸馏最严重的因果泄漏之一。

诊断信号	修复策略
离线 imitation 很高但在线 agent success 很低；重放轨迹能做对，真实环境不能做；训练样本拼接方式包含未来状态	构建 causal masking / step-wise dataset；每一步仅暴露 history_t；用在线 replay 验证训练样本可复现性

验证指标： offline-online gap；causal replay pass rate；leakage audit

15. 只蒸馏成功轨迹造成 Survivorship Bias [P1]

完美轨迹不包含真实环境中最重要的恢复能力：工具失败、超时、找不到文件、编译错误、搜索结果无关。

诊断信号	修复策略
第一次工具失败后行为失控；重复调用同一失败工具；无法撤销或替换策略	保留 failure + recovery trajectories；人工构造工具错误与超时；单独评估 recovery benchmark

验证指标： recovery success；retry efficiency；failure-type coverage

16. Reward / Verifier 被投机利用 [P1]

如果 verifier 只检查表面条件，Student 会学会满足评分器而非真正解决任务，尤其在 Agent 和代码任务中常见。

诊断信号	修复策略
reward 上升但真实任务成功率不涨；出现固定模板绕过检查； 单一 verifier 与人工判断分歧大	多层 verifier：规则+执行+模型评估；设置 adversarial checks； 周期性人工抽检 reward hacking

验证指标： reward-real success correlation；human disagreement；adversarial pass rate

07 遗忘、对齐与部署 Gotchas

17. Catastrophic Forgetting / Safety Regression / Quantization Gap [P1]

领域蒸馏可能推高 coding/math 等目标能力，却同时损害通用 QA、中文、指令跟随与安全；蒸馏后的 activation 分布又可能在 INT4/INT8 下进一步恶化。

诊断信号	修复策略
目标域上涨但通用 benchmark 下跌；拒答/边界行为回退；FP16 好但量化 serving 明显掉点	加入 general replay / alignment replay；训练后做全能力回 归；最终评测必须跑真实 serving stack

验证指标： target uplift vs general regression；safety regression；FP16-INT4 gap

08 推荐的工业化 Distillation Pipeline

目标是最大化“每个训练 token 的有效信息增益”，并保证数据可验证、可追溯。

阶段	输入	核心动作	关键 Gate
1. Traffic Profiling	真实流量 / benchmark / 任务池	分桶：难度、长度、语言、工具 状态、错误类型	训练分布覆盖率
2. Student Baseline	当前 Student	跑基线并记录置信度、失败类型	找出不会的区域
3. Teacher Rollout	困难样本	多采样、不同温度、必要时多 Teacher	候选多样性
4. Verification	候选答案/轨迹	执行验证、规则、模型 verifier、 环境结果	仅保留可证伪/可验证高质量样本
5. Data Selection	verified samples	dedup、reweight、 curriculum、hard mining	有效样本量与覆盖
6. Training	混合数据	domain + general replay + alignment replay	目标能力 ↑ 且通用能力不显著退 化
7. Production-like Eval	量化/serving/真实 prompt	线上堆栈、长上下文、工具失 败、压力测试	offline-online gap 可接受
8. Loop	新失败样本	持续 disagreement mining	能力增益可持续

最值得投入的模块

Verifier 与 Hard Example Mining 往往比“再生成 10 倍 synthetic data”更有价值。前者提高标签正确率，后者提高每个 token 的信息密度。

09 评测矩阵：不要只看一个 Benchmark

维度	至少需要的评测	典型指标
Capability	目标任务 + 私有 holdout	Accuracy / Pass@k / Task Success
Generalization	跨模板、跨领域、噪声输入	OOD performance / robustness
Process	中间决策与轨迹	step accuracy / trajectory success
Calibration	不确定性与拒答	ECE / Brier / abstention accuracy
Agent	工具调用、失败恢复、长链	tool precision / recovery / online success
Safety	边界行为与拒答	policy regression / unsafe completion rate
Efficiency	token、延迟、吞吐	accuracy/token / latency / TPS
Deployment	INT4/INT8 + serving stack	FP16-serving gap / long-context degradation

评测红旗

如果“公开 benchmark 大涨、私有 holdout 小涨或不涨”，优先怀疑 contamination、模板记忆或 style imitation，而不是立刻宣布 reasoning 能力提升。

10 上线前检查清单

可直接用于蒸馏项目的设计评审 / Go-NoGo Gate。

类别	检查项	状态
数据	是否统计真实生产输入的长度、语言、噪声、模糊度和工具状态分布？	☐ Pass ☐ Fail ☐ N/A
数据	是否做 exact / fuzzy / semantic dedup？	☐ Pass ☐ Fail ☐ N/A
数据	是否扫描 benchmark contamination 与 template leakage？	☐ Pass ☐ Fail ☐ N/A
Teacher	Teacher 输出是否有自动 verifier？哪些类别仍只能依赖模型判断？	☐ Pass ☐ Fail ☐ N/A
Teacher	是否使用多采样或多 Teacher 以降低单一偏	☐ Pass ☐ Fail ☐ N/A

类别	检查项	状态
	置?	
Student	是否做过 Student 可学习性 pilot，而不是直接大规模训练?	☐ Pass ☐ Fail ☐ N/A
Reasoning	是否区分关键决策信息与冗余 Raw CoT?	☐ Pass ☐ Fail ☐ N/A
Agent	每个 action_t 是否严格只看到 history_t?	☐ Pass ☐ Fail ☐ N/A
Agent	训练集是否包含 failure + recovery trajectory?	☐ Pass ☐ Fail ☐ N/A
Training	是否有 general replay / alignment replay 防止遗忘?	☐ Pass ☐ Fail ☐ N/A
Evaluation	是否有私有 holdout、OOD、robustness 与 calibration 评测?	☐ Pass ☐ Fail ☐ N/A
Deployment	是否在最终量化、serving、长上下文与真实工具链上复测?	☐ Pass ☐ Fail ☐ N/A
Governance	是否记录每条数据的 provenance、teacher、verifier、版本与过滤原因?	☐ Pass ☐ Fail ☐ N/A

11 常见 Anti-Patterns 与替代方案

Anti-Pattern	为什么不够	更好的替代
随机生成大量 prompt → Teacher → SFT	大量 easy/duplicate 数据，错误未验证	真实流量分桶 + disagreement mining + verifier
只用 temperature=0 单采样	mode collapse，缺乏多解与鲁棒性	N-sample + best-of-N + diverse positives
Raw CoT 全量保留	训练冗长、错误或事后解释	关键决策点压缩 + step verification
只看目标 benchmark	可能污染、记忆、遗忘其他能力	private holdout + general + OOD + deployment eval
只蒸馏成功 Agent trajectory	没有失败恢复能力	success + failure + recovery 混合轨迹
FP16 checkpoint 通过就上线	量化与 serving 改变分布	在最终 INT4/INT8、长上下文、工具环境复测

12 如何判断“这次蒸馏是否真的有效”

推荐判据

一次蒸馏只有同时满足以下三点，才更接近“能力迁移”而不是“风格模仿”：① 私有 holdout 与 OOD 同时提升；② 关键中间决策或在线任务成功率提升；③ 在真实 serving stack 下仍保持收益。

现象	更可能的解释	下一步
输出更像 Teacher，但准确率不涨	style / format imitation	缩短或过滤解释，转向 hard samples
公开集大涨、私有集不涨	contamination / template leakage	清洗数据并换 private holdout
短任务涨、长链不涨	process supervision 不足	加入关键决策与 recovery trajectory
目标域涨、通用域掉很多	catastrophic forgetting	增加 replay，降低 domain 数据权重
FP16 涨、INT4 掉	quantization-sensitive distribution	QAT/重新校准/部署态训练与评测
离线轨迹很好、在线 agent 差	future observation leakage / environment mismatch	做 causal dataset audit 与 online replay

13 最终结论

模型蒸馏最容易被低估的事实是：训练算法往往不是主要瓶颈，数据选择与验证机制才是。

- 不要把 Teacher 输出默认当作真值；对可执行任务，优先使用环境、编译器、测试或符号验证。
- 不要平均地蒸馏所有样本；优先蒸馏 Student 的知识缺口与真实线上失败。
- 不要把 Raw CoT 当作天然高质量过程监督；蒸馏关键决策状态比蒸馏所有 reasoning token 更稳。
- Agent 轨迹必须满足严格因果性，并包含失败恢复，而不仅是成功演示。
- 评测必须覆盖私有 holdout、OOD、通用能力、安全、校准和真实部署堆栈。
- 最终目标不是“复刻 Teacher”，而是用有限 Student 容量最大化可靠、可泛化、可部署的能力。

最值得记住的一句话

Distillation $\neq$ Teacher Output SFT。成熟系统更像一个持续运行的 Data Engine：发现知识缺口 $\rightarrow$ 多样 Teacher rollout $\rightarrow$ 自动验证 $\rightarrow$ 去重与课程化 $\rightarrow$ 训练 $\rightarrow$ 真实环境评测 $\rightarrow$ 回流新的困难样本。